Guía Gamificada de U1T2

25 Minutos

Modelado con PlantUML: 2 diagramas de casos de uso nivel 0 + 1 diagrama de clases de análisis

Caso: aplicativo móvil para seguimiento a egresados ESPE – IASA-I

Dirigido a	Estudiantes de 6to nivel de la carrera de Software con base previa en Ingeniería de Requisitos.
Modalidad	Trabajo colaborativo en equipos de 3 a 4 integrantes.
Producto final	Dos diagramas de casos de uso nivel 0 y un diagrama de clases de análisis construidos con sintaxis PlantUML.

Idea central del reto: traducir requisitos funcionales del proyecto a modelos UML claros, trazables y coherentes, usando PlantUML como herramienta de representación textual.

1. Propósito del taller

Adaptar la guía gamificada al modelado de artefactos de análisis para el proyecto de seguimiento a egresados. El equipo trabajará como si fuera una célula de análisis responsable de transformar necesidades del caso en modelos UML iniciales listos para discusión técnica.

2. Base conceptual del reto

El caso del proyecto plantea una necesidad institucional: mantener contacto continuo con egresados, actualizar información personal, difundir oportunidades, apoyar capacitación y recopilar retroalimentación para la mejora académica.

Desde Ingeniería de Requisitos, el taller se enfoca en tres preguntas clave:

- ¿Quién interactúa con el sistema? → actores.
- ¿Qué objetivos persigue cada actor? → casos de uso.
- ¿Qué conceptos del dominio participan y cómo se relacionan? → clases de análisis.

3. Objetivos de aprendizaje

Lo que ya conocen	Lo que practicarán hoy	Evidencia esperada
Actores y necesidades	Identificación de actores primarios y secundarios	Listado depurado de actores
Requisitos funcionales	Conversión de RF a casos de uso nivel 0	2 diagramas de CU coherentes
Trazabilidad	Relación entre objetivo del negocio y funcionalidad	Justificación breve por equipo

Análisis de dominio	Extracción de entidades, atributos y relaciones	1 diagrama de clases de análisis

4. Modelos que debe construir cada equipo

- Diagrama 1: Casos de uso nivel 0 orientado al actor Egresado.
- Diagrama 2: Casos de uso nivel 0 orientado al actor Unidad de Seguimiento a Graduados / Administrador.
- Diagrama 3: Diagrama de clases de análisis del dominio principal del sistema.

Sugerencia didáctica: cada diagrama debe responder a una parte clara del problema. No se trata de dibujar todo, sino de modelar lo esencial de la primera versión del sistema.

5. Mecánica gamificada

Insignia	Cómo se obtiene	Puntos
Radar de Actores	Identifica actores correctos y evita redundancias.	3
Arquitecto de Casos de Uso	Construye 2 diagramas nivel 0 con nombres claros y relaciones válidas.	5
Modelador del Dominio	Propone clases de análisis pertinentes con relaciones coherentes.	5
Sintaxis PlantUML Segura	Escribe código PlantUML legible y consistente.	3

6. Secuencia del taller (25 minutos)

Tiempo	Fase	Actividad del equipo	Resultado esperado
0-3 min	Activación	Asignar roles: coordinador, analista de requisitos, modelador PlantUML y vocero. Leer el caso base.	Equipo organizado.
3-8 min	Ronda 1	Identificar actores y agrupar funcionalidades por objetivo del usuario.	Mapa de actores y objetivos.
8-14 min	Ronda 2	Construir el primer diagrama de casos de uso nivel 0 orientado al Egresado.	CU nivel 0 – Actor Egresado.
14-19 min	Ronda 3	Construir el segundo diagrama de casos de uso nivel 0 orientado a la Unidad de Seguimiento / Administrador.	CU nivel 0 – Actor institucional.
19-23 min	Ronda 4	Extraer clases del dominio, atributos esenciales y relaciones de análisis.	Diagrama de clases de análisis.
23-25 min	Cierre	Validar coherencia entre requisitos, actores y clases. Exposición breve.	Síntesis y retroalimentación.

7. Desarrollo del reto por rondas

Ronda 1. Radar de actores y objetivos

- Preguntas guía: ¿quién usa la app?, ¿quién administra la información?, ¿qué acciones son visibles para cada actor?
- Evitar errores frecuentes: confundir actor con cargo interno irrelevante, repetir el mismo actor con nombres distintos o convertir pantallas en casos de uso.

Ronda 2 y 3. Casos de uso nivel 0

- Seleccionar casos de uso principales, no subprocesos de bajo nivel.
- Nombrar los casos de uso con verbo en infinitivo: Actualizar perfil, Consultar oportunidades, Registrar retroalimentación, Gestionar noticias, Publicar oportunidades, Generar reportes.
- Usar relaciones <<include>> o <<extend>> solo cuando realmente aporten claridad.

Ronda 4. Diagrama de clases de análisis

- Extraer clases del dominio a partir de sustantivos relevantes del caso: Egresado, Perfil, Oportunidad, Notificación, Capacitación, Encuesta, Retroalimentación, UsuarioAdministrador.
- No modelar aún detalles de implementación como pantallas, botones, APIs o clases técnicas.
- Priorizar relaciones de asociación, agregación o composición solo si son justificables desde el análisis.

8. Caso base del taller

Caso: El Club de Suboficiales requiere una aplicación web para planificar y dar seguimiento a eventos sociales y deportivos. El sistema debe permitir organizar actividades especiales y mensuales, gestionar propuestas de socios, controlar asistencia, administrar recursos económicos por parte del tesorero y generar reportes para apoyar la toma de decisiones de la directiva.

9. Entregable del equipo

CU-003 Consultar calendario de eventos

Diagrama de caso de uso

```
@startuml
left to right direction
actor "Socio" as socio
actor "Secretario" as secretario
actor "Presidente" as presidente
actor "Tesorero" as tesorero
rectangle "Sistema de Gestión de Eventos" {
    usecase "Consultar calendario de eventos" as UC1
    usecase "Filtrar por tipo (social/deportivo)" as UC2
    usecase "Configurar recordatorio" as UC3
    usecase "Ver detalle del evento" as UC4
    UC2 ..> UC1 : <<extend>>
    UC3 ..> UC1 : <<extend>>
```

```
    UC1 ..> UC4 : <<include>>
}
socio --> UC1
secretario --> UC1
presidente --> UC1
tesorero --> UC1
@enduml
```

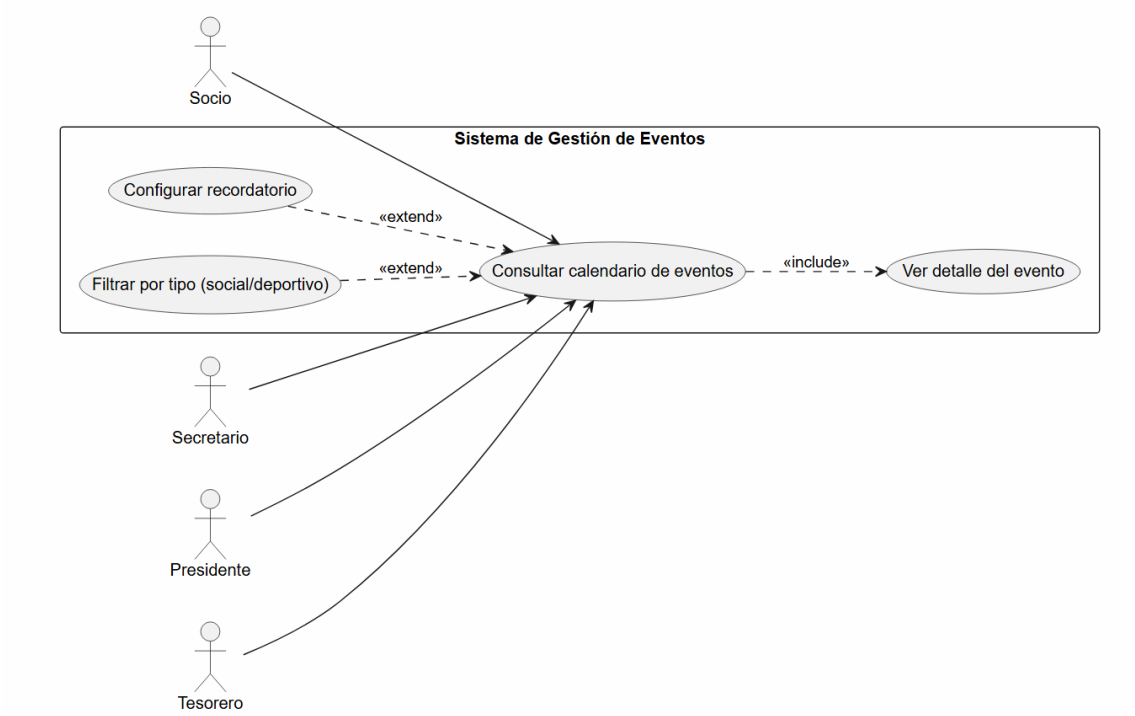
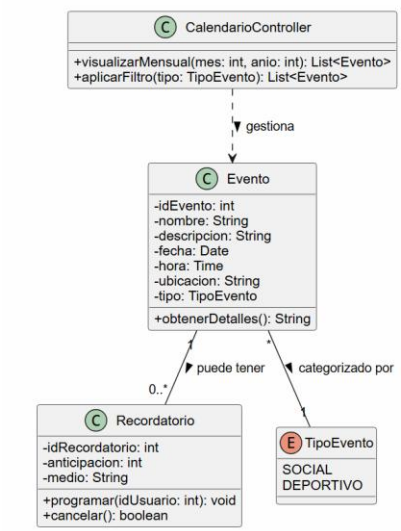


Diagrama de clases

```
@startuml
skinparam classAttributeIconSize 0
class CalendarioController {
    + visualizarMensual(mes: int, anio: int): List<Evento>
    + aplicarFiltro(tipo: TipoEvento): List<Evento>
}
class Evento {
    - idEvento: int
    - nombre: String
    - descripcion: String
    - fecha: Date
    - hora: Time
    - ubicacion: String
    - tipo: TipoEvento
    + obtenerDetalles(): String
}
class Recordatorio {
    - idRecordatorio: int
    - anticipacion: int
    - medio: String
    + programar(idUsuario: int): void
    + cancelar(): boolean
}
```

```
enum TipoEvento {
    SOCIAL
    DEPORTIVO
}
CalendarioController ..> Evento : gestiona >
Evento "1" -- "0..*" Recordatorio : puede tener >
Evento "*" -- "1" TipoEvento : categorizado por >
@enduml
```



CU-004: Proponer Evento

Diagrama de caso de uso

```
@startuml
left to right direction
actor "Socio" as socio
actor "Secretario" as secretario
rectangle "Sistema de Gestión de Eventos" {
    usecase "Proponer nuevo evento" as UC_PROP
    usecase "Adjuntar documentos/propuesta" as UC_ADJ
    usecase "Notificar a directiva" as UC_NOTIF
    usecase "Validar requisitos mínimos" as UC_VAL
    socio --> UC_PROP
    UC_PROP ..> UC_VAL : <<include>>
    UC_PROP ..> UC_NOTIF : <<include>>
    UC_ADJ ..> UC_PROP : <<extend>>
    UC_NOTIF -- secretario
}
@enduml
```

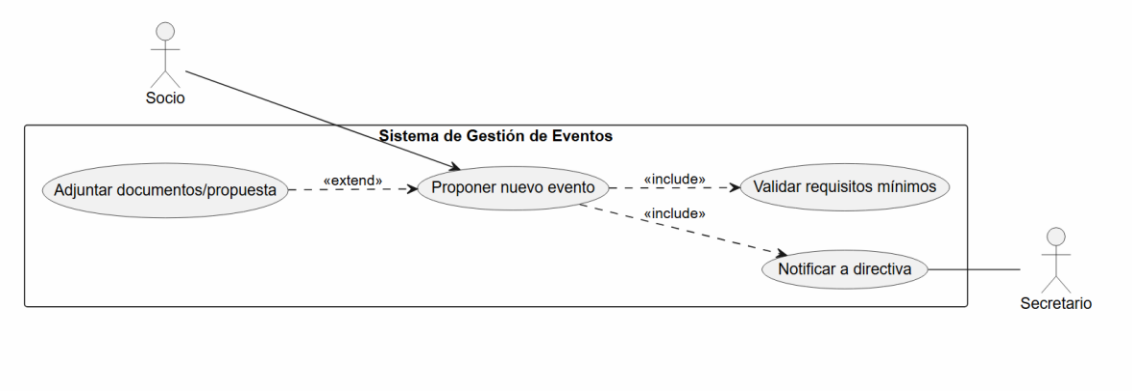
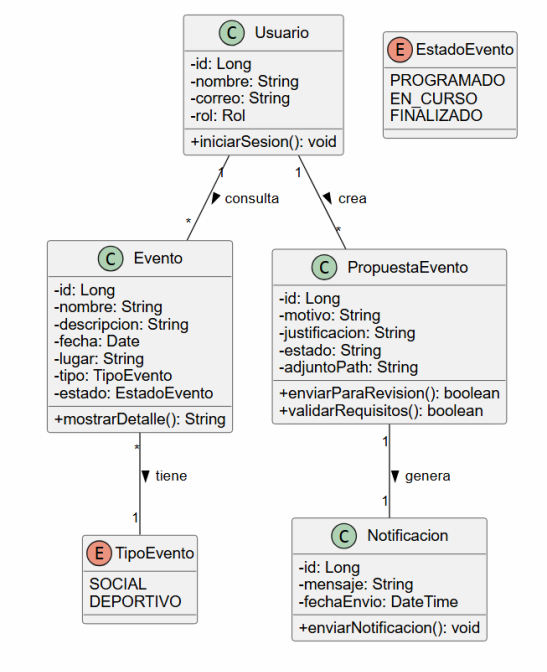


Diagrama de clases

```
@startuml
skinparam classAttributeIconSize 0
class Usuario {
    - id: Long
    - nombre: String
    - correo: String
    - rol: Rol
    + iniciarSesion(): void
}
class Evento {
    - id: Long
    - nombre: String
    - descripcion: String
    - fecha: Date
    - lugar: String
    - tipo: TipoEvento
    - estado: EstadoEvento
    + mostrarDetalle(): String
}
class PropuestaEvento {
    - id: Long
    - motivo: String
    - justificacion: String
    - estado: String
    - adjuntoPath: String
    + enviarParaRevision(): boolean
    + validarRequisitos(): boolean
}
class Notificacion {
    - id: Long
    - mensaje: String
    - fechaEnvio: DateTime
    + enviarNotificacion(): void
}
enum TipoEvento {
    SOCIAL
    DEPORTIVO
}
enum EstadoEvento {
    PROGRAMADO
    EN_CURSO
    FINALIZADO
}
Usuario "1" -- "*" Evento : consulta >
```

```
Usuario "1" -- "*" PropuestaEvento : crea >
PropuestaEvento "1" -- "1" Notificacion : genera >
Evento "*" -- "1" TipoEvento : tiene >
@enduml
```



10. Plantilla rápida para completar en clase

Actores principales	
Casos de uso del Egresado	
Casos de uso de la Unidad / Administrador	
Clases candidatas del dominio	
Relaciones principales entre clases	
Observaciones de trazabilidad	

11. Mini sintaxis PlantUML esencial

Ejemplo 1. Caso de uso nivel 0

```
@startuml
left to right direction
actor Egresado
rectangle "App de Seguimiento a Egresados" {
  usecase "Actualizar perfil" as UC1
  usecase "Consultar oportunidades" as UC2
```

```
  usecase "Registrar retroalimentación" as UC3
}
Egresado --> UC1
Egresado --> UC2
Egresado --> UC3
@enduml
```

Ejemplo 2. Diagrama de clases de análisis

```
@startuml
class Egresado {
  id: int
  nombre: String
  correo: String
}
class Perfil {
  telefono: String
  direccion: String
}
class Oportunidad {
  id: int
  titulo: String
  tipo: String
}
Egresado *-- Perfil
Egresado --> Oportunidad : consulta
@enduml
```

12. Criterios de evaluación rápida

Criterio	Qué se observa	Puntaje
Comprensión del caso	Los actores y objetivos corresponden al problema del proyecto.	5
Calidad de casos de uso	Los 2 diagramas nivel 0 son claros, consistentes y sin redundancias.	5
Calidad del diagrama de clases	Las clases de análisis representan el dominio y sus relaciones esenciales.	5
Uso de PlantUML y comunicación	La sintaxis es legible y la explicación del equipo es técnica y breve.	5

13. Cierre sugerido del docente

Mensaje de cierre: modelar bien no es dibujar mucho, sino representar con precisión lo que el sistema debe hacer y los conceptos clave que lo sostienen. Si los requisitos están claros, PlantUML permite convertirlos en artefactos trazables, discutibles y técnicamente útiles.

Documento adaptado para el caso de seguimiento a egresados ESPE – IASA-I y orientado al uso de PlantUML en Análisis y Diseño de Software.